

Introduction to Software Architecture

Diagramming For Architects

- Visual Representation: Diagrams provide a clear and structured way to visualize complex systems, aiding in understanding and communication.
- Communication Tool: They help convey ideas effectively among stakeholders, including developers, business analysts, and clients.
- System Design: Enable architects to conceptualize, design, and document software systems at various abstraction levels.
- Problem Identification: Facilitate the analysis of dependencies and potential issues within the system.
- Alignment with Requirements: Ensure that the system's design aligns with business and technical requirements.
- Collaboration Enhancement: Foster better collaboration among team members by serving as a common reference point.
- Scalability and Decision-Making: Aid in planning for scalability and making informed architectural decisions.
- Audience Bridging: Translate complex ideas into visuals that bridge the gap between technical and non-technical stakeholders.

High Level Architecture Diagrams

Purpose: A high-level architecture diagram for software is a visual representation of the overall structure and major components of a software system, showing how they interact with each other without going into low-level implementation details

Use Cases:

- Communicate system design to stakeholders (technical and non-technical).
- Identify major modules, subsystems, and interfaces.
- Provide a blueprint for development and integration.

Component Diagrams

Purpose: Depicts the structural organization of components within a system and their relationships.

Use Cases: Useful for breaking down large systems into manageable components.

Sequence Diagrams

Purpose: Illustrates interactions between objects in the order they occur.

Use Cases: Useful for capturing complex business workflows and process flows.

Hands On Diagrams

Clean Architecture

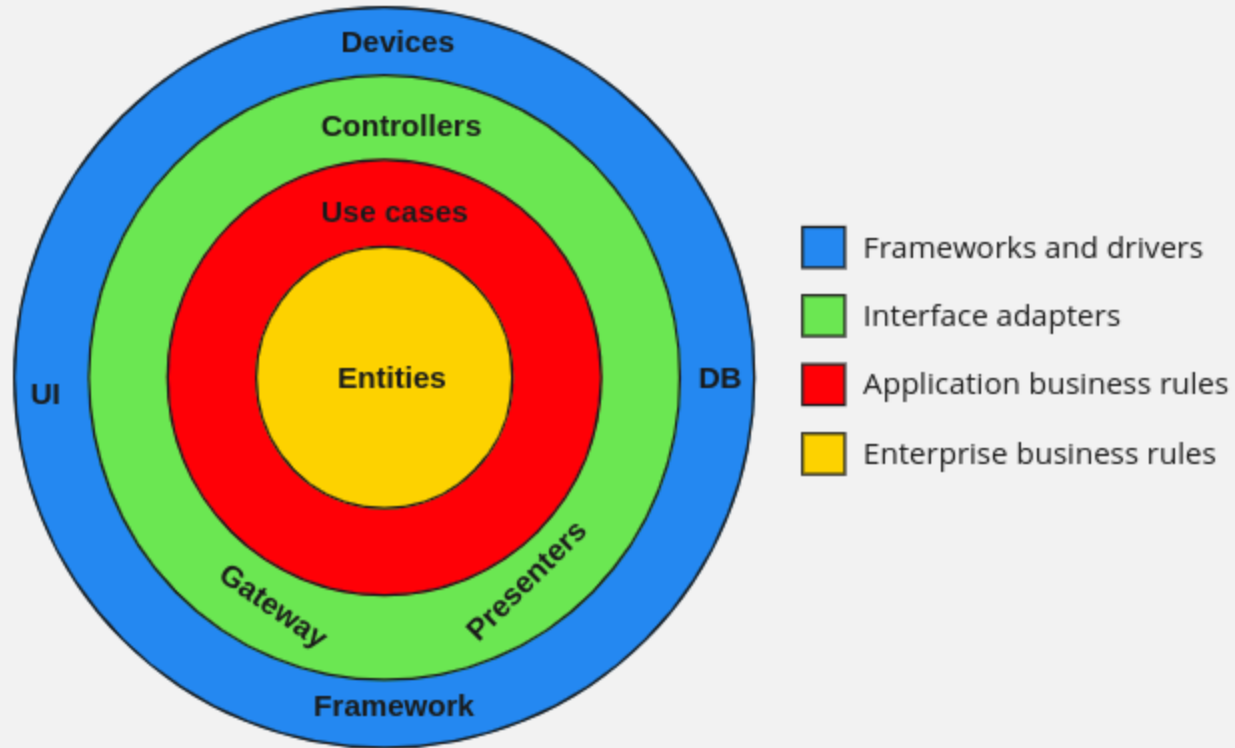
Clean Architecture is a software design approach introduced by Robert C. Martin (Uncle Bob) that emphasizes separation of concerns and independence of different layers in an application. The goal is to create a system that is maintainable, testable, scalable, and adaptable to changing requirements over time.

Key Principles of Clean Architecture

- **Independence:**
 - Framework-independent: The system should not be tightly coupled to any specific frameworks or libraries.
 - UI-independent: The application's core logic should remain unaffected by changes in the user interface.
 - Database-independent: Business logic should not depend on a particular database or data access technology.
 - Testable: The core of the system should be easily testable without requiring external dependencies like databases or UI.
- **Separation of Concerns:**
 - The architecture enforces clear separation between different responsibilities, such as business rules, data access, and external interfaces.
- **Dependency Rule:**
 - Inner layers must not depend on outer layers. Instead, dependencies should always point inward, toward the core logic.

Structure of Clean Architecture

- **Entities (Domain - Core Business Rules):**
 - Represent the business objects and rules.
 - Should be independent of any external systems or frameworks.
 - Examples: Validation rules, domain models.
- **Use Cases (Application - Application Business Rules):**
 - Contain the application-specific logic that coordinates the flow of data between entities and interfaces.
 - Examples: Workflow orchestrations, input validation for specific use cases.
- **Interface Adapters (Presentation -Delivery Mechanisms):**
 - Translate data between the core logic (use cases) and external systems (e.g., UI, databases, APIs).
 - Examples: Controllers, Presenters, Gateways.
- **Frameworks and Drivers (Infrastructure - External Systems):**
 - Contain the actual implementation details for frameworks, libraries, databases, and APIs.
 - This is the most volatile layer and should be interchangeable without impacting the core logic.



Source: Adapted from MARTIN (2017)

Advantages of Clean Architecture

- Maintainability: Clear boundaries make it easier to modify one part of the system without affecting others.
- Testability: Core business logic can be tested in isolation without relying on external dependencies.
- Scalability: The modular structure allows the system to grow and adapt to new requirements or technologies.
- Framework Agnostic: The core logic is not tied to a specific framework, making migrations or upgrades easier.

Hands On

The system shall allow a new user to register by providing their name, DOB, email, and password. Upon successful registration, the user should receive a confirmation email. User must be 18 years or older to register.

Framework Design

Creating a strong framework for a large, complex application ensures that the application's control flow and core behavior remain consistent and cannot be inadvertently altered by developers.

Define the Core Framework Responsibilities

- Establish the control flow: The framework manages how different parts of the application interact.
- Handle cross-cutting concerns: Centralized logging, security, exception handling, and transaction management.
- Provide a standardized lifecycle: Define stages like initialization, execution, and termination.
- Ensure consistency: Enforce uniform behavior across all application modules.

Use Interface-Based Programming

- Expose specific interfaces for developers to implement.
- Ensure interfaces represent isolated responsibilities (e.g., `IDataProcessor`, `IEventHandler`).
- Use contracts to enforce behavior, ensuring developers adhere to predefined requirements.

Control the Execution Flow

Use inversion of control (IoC): The framework controls the flow, invoking developer-implemented interfaces at the appropriate times.

Implement dependency injection to provide the necessary resources to developer-created components.

Use template method design pattern: The framework defines the skeleton of operations, and developers fill in specific steps.

Restrict Direct Modifications

- Keep the core logic sealed: Prevent developers from modifying core framework classes.
- Use final classes or private constructors to enforce restrictions.
- Use code reviews or automated tools to ensure compliance with framework standards.

Provide Extensibility Points

- Define extension points like hooks or plugins where custom behavior is allowed.
- Use configuration-driven design: Allow customization through external configuration (e.g., YAML, JSON), not by changing code.
- Support event-driven programming: Developers write handlers for framework-triggered events.

Automate Compliance with the Framework

- Provide pre-built tools like scaffolding, code generators, or templates to guide development.
- Integrate linting and static code analysis tools to check adherence to framework guidelines.
- Include comprehensive unit tests within the framework to ensure predictable behavior.

Enforce Standardized Error Handling

Centralize error handling and logging within the framework.

Ensure all exceptions are handled by the framework, with developers allowed to log or annotate only.

Hands on ETL Process

Write an ETL process for migrating data from Postgres to MongoDB